Project Documentation

Name: ARIJIT GUHA

Introduction

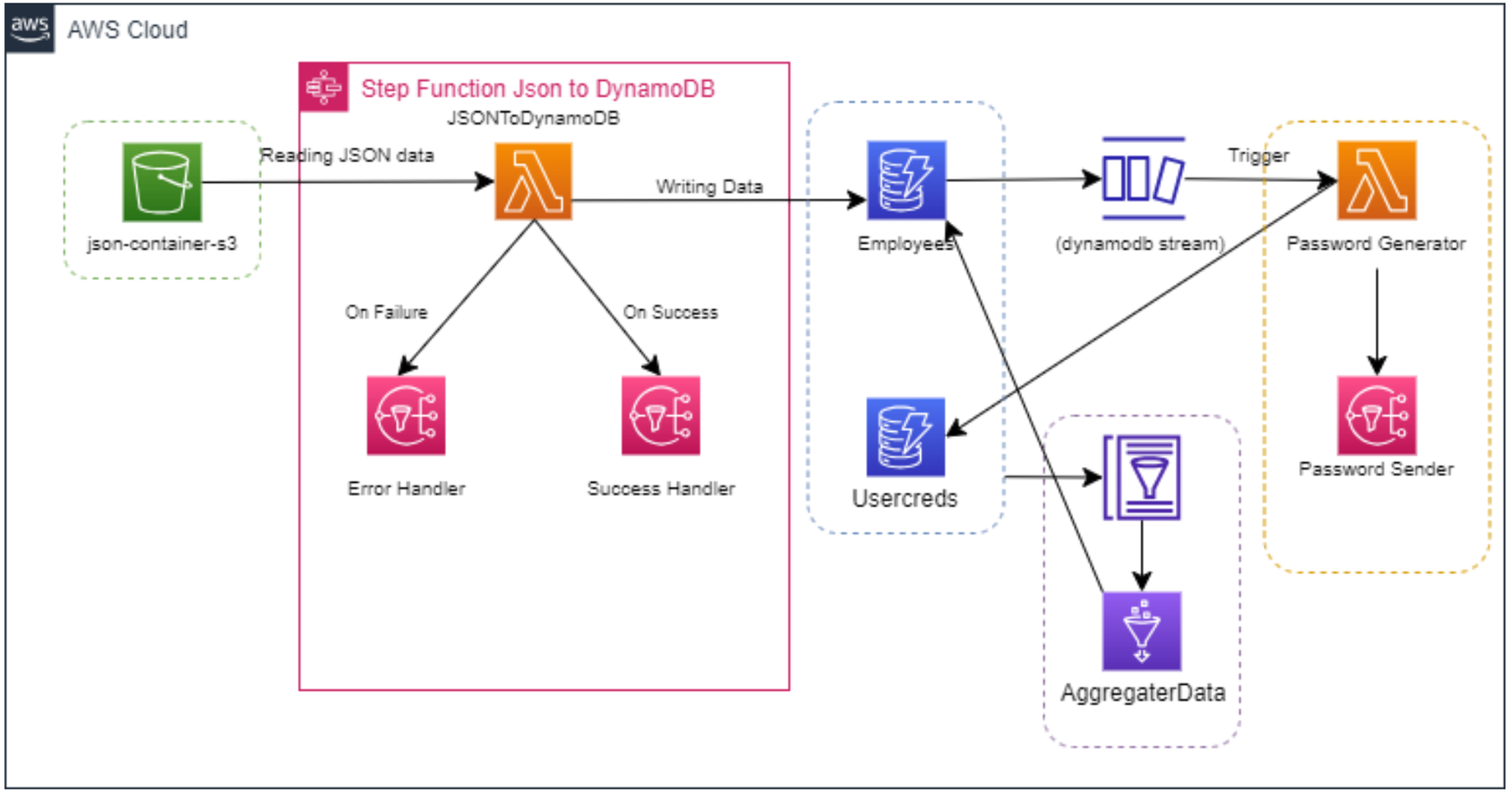
This document presents the project documentation for an AWS project with Power BI integration. The project aims to create a workflow using AWS services, including Step Functions, Lambda, DynamoDB, SNS, and Glue. The project also involves visualizing and analyzing data in Power BI. This documentation outlines the project's objectives, architecture, data flow, and implementation details.

Project Scope

The project focuses on the following components and tasks:

1.Creation of a Step Function to orchestrate the workflow.
2.Passing a file path name (bucket/data.json) to trigger subsequent actions.
3.Reading the file using a Lambda function and saving the data into a DynamoDB table.
4.Sending success/failure notifications to a specific email address using SNS.
5.Implementing a DynamoDB stream to trigger a Lambda function for generating unique usernames and passwords, sending emails, and saving data in the usercreds table.
6.Performing batch processing using Glue (Pyspark) to combine data from the employees table and the usercreds table on a daily basis.

Architecture Diagram:



Data Sources

The project utilizes the following data sources:
- File: The project expects a file named abc.json located in an AWS S3 bucket.
- DynamoDB Tables: Two tables are used - employees and usercreds. The employees table stores employee-related data, and the usercreds table contains generated usernames and passwords.

Implementation Steps

1. Step Function:
   - Create a Step Function using AWS Step Functions service.
   - Define the state machine to orchestrate the workflow based on the project requirements.
2. Lambda Function - Read File and Save to DynamoDB:
   - Create a Lambda function that reads the file from the provided path (bucket/abc.json).
   - Parse the file data and save it to the DynamoDB employees table.
3. SNS Notification:
   - Configure an SNS topic to send success/failure notifications.
   - Subscribe a specific email address to the SNS topic to receive notifications.
4. DynamoDB Stream and Lambda Function - Generate User Credentials:
   - Enable DynamoDB Stream on the employees table.
   - Create a Lambda function triggered by the DynamoDB Stream.
   - Implement the function to generate unique usernames and passwords, send emails with credentials to your mail id, and save the data to the usercreds table.
5. Glue Batch Processing:
   - Configure an AWS Glue job using Pyspark to process data from the usercreds and employees tables, and Schedule the Glue job on daily basis.

Source Code:

JSONToDynamoDB:

```
import json
import boto3
import os
s3_client = boto3.client("s3")
dynamodb = boto3.resource("dynamodb")
class S3Error(Exception):
    "Error in S3 bucket reading"
    pass
def save_to_dynamodb(id,name,salary,age,table):
    return table.put_item(Item={'id': id,
                'employee_name': name,
                'employee_salary': salary,
                'employee_age': age,
                'profile_image': ""})
def lambda_handler(event, context):
    try:
        table_name = "employees"
        path = event['path']
        file_name = os.path.basename(path)
        bucket_name = os.path.dirname(path)
        table = dynamodb.Table(table_name)
        response = s3_client.get_object(Bucket=bucket_name, Key=file_name)
        data =  response['Body'].read().decode('utf-8')
        json_content = json.loads(data)
        print(json_content)
        for i in json_content['employees']:
            id = i['id']
            name = i['employee_name']
            salary=i['employee_salary']
            age = i['employee_age']
            save_to_dynamodb(id,name,salary,age,table)
        return "Hey the json file is uploaded to your dynamodb Table"
    except:
        raise S3Error
```

PasswordGenerator:

```
import json
import boto3
import random
import string
sns = boto3.client("sns")
dynamodb = boto3.client("dynamodb")

def generate_random_string(length):
    letters_and_digits = string.ascii_letters + string.digits
    return ''.join(random.choice(letters_and_digits) for _ in range(length))

def lambda_handler(event, context):
    try:
        record = event['Records'][0]['dynamodb']['NewImage']
        id = record['id']['N']
        name= record['employee_name']['S']
        age = record['employee_age']['N']

        username=name+str(age)
        username = username.replace(" ", "")
        password = generate_random_string(10)

        message = "Your Username is "+username+" and your password is: "+password
        dynamodb.put_item(TableName='employees_userid', Item={'id':{'N':id},'username':
{'S':username},'password':{'S':password}})
        response = sns.publish(TopicArn='arn:aws:sns:ap-south-
1:931700653651:PasswordSender',Message=message)
        print("Message published")
        print("working?")
    except:
        return -1
```

Employees Data

employee_age	employee_name	employee_salary	id	profile_image
{'N': '61'}	{'S': 'Tiger Nixon'}	{'N': '320800'}	{'N': '1'}	{'S': ''}
{'N': '63'}	{'S': 'Garrett Winters'}	{'N': '170750'}	{'N': '2'}	{'S': ''}
{'N': '66'}	{'S': 'Ashton Cox'}	{'N': '86000'}	{'N': '3'}	{'S': ''}
{'N': '22'}	{'S': 'Cedric Kelly'}	{'N': '433060'}	{'N': '4'}	{'S': ''}
{'N': '33'}	{'S': 'Airi Satou'}	{'N': '162700'}	{'N': '5'}	{'S': ''}

Usercreds Data

id	password	username
{'N': '5'}	{'S': 'roVvt1ILQI'}	{'S': 'AiriSatou33'}
{'N': '4'}	{'S': '545kb7o0DU'}	{'S': 'CedricKelly22'}
{'N': '3'}	{'S': 'VwY48I3lRW'}	{'S': 'AshtonCox66'}
{'N': '2'}	{'S': '3g7axGBe7l'}	{'S': 'GarrettWinters63'}
{'N': '1'}	{'S': 'n0X1akoX9J'}	{'S': 'TigerNixon61'}

Processed Employees data by Glue

employee_age	employee_name	employee_salary	password	id	profile_image	username
{'N': '22'}	{'S': 'Cedric Kelly'}	{'N': '433060'}	{'S': '545kb7o0DU'}	{'N': '4'}	{'S': ''}	{'S': 'CedricKelly22'}
{'N': '33'}	{'S': 'Airi Satou'}	{'N': '162700'}	{'S': 'roVvt1ILQI'}	{'N': '5'}	{'S': ''}	{'S': 'AiriSatou33'}
{'N': '61'}	{'S': 'Tiger Nixon'}	{'N': '320800'}	{'S': 'n0X1akoX9J'}	{'N': '1'}	{'S': ''}	{'S': 'TigerNixon61'}
{'N': '63'}	{'S': 'Garrett Winters'}	{'N': '170750'}	{'S': '3g7axGBe7l'}	{'N': '2'}	{'S': ''}	{'S': 'GarrettWinters63'}
{'N': '66'}	{'S': 'Ashton Cox'}	{'N': '86000'}	{'S': 'VwY48I3lRW'}	{'N': '3'}	{'S': ''}	{'S': 'AshtonCox66'}